



THE AUTO-RFP ENGINE

Turn one Google Sheet into an automated pricing-request machine. Email every vendor a private link, collect prices automatically, and level the bids in one grid.

WHAT'S INSIDE

- 01 What this system does
- 02 What you'll need before you start
- 03 Setting up your sheet structure
- 04 Installing the script (code included)
- 05 The first authorization (don't panic)
- 06 Deploying the vendor pricing web form
- 07 Sending your first batch of RFPs
- 08 Reading the Tracking tab
- 09 Nudging non-responders
- 10 Reading the Comparison tab (bid leveling)
- 11 Troubleshooting & limits
- 12 How to customize this

Prepared by TakeoffMonkey for our contractor customers. Everything runs inside Google for free — no third-party software, no subscriptions.

SECTION 01

WHAT THIS SYSTEM DOES

This system turns a Google Sheet into an automated pricing-request (RFP) machine. You paste in your materials list and your vendor list, click one button, and it emails every vendor a private link. Each vendor clicks their link, sees a clean web form with your materials already loaded, types in their prices, and clicks submit. Their pricing lands back in your sheet automatically, and you get an email copy too. A Tracking tab shows you who you sent to, who responded, and when. A Comparison tab levels every vendor's pricing side by side. A second button nudges anyone who hasn't replied after two days.

The flow in plain English: You click Send → vendors get personalized emails with unique links → vendor opens link → fills out web form → clicks submit → pricing flows back into your Tracking and Comparison



tabs → you click Nudge to chase stragglers.

Everything runs inside Google for free. No third-party software, no subscriptions, no per-email cost beyond your normal Gmail sending limits.

SECTION 02

WHAT YOU'LL NEED BEFORE YOU START

A Google account (personal Gmail or Google Workspace). One Google Sheet, which will hold everything: your materials, your vendors, your email settings, and your tracking. About 20 minutes for the one-time setup. That's it. You do not need to know how to code — you'll copy and paste the code exactly as written.

One important limit to know up front: a personal Gmail account can send to about 100 recipients per day. A Google Workspace account can send to about 1,500 per day. If you have a huge vendor list, send in batches across multiple days, or use a Workspace account.

SECTION 03

SETTING UP YOUR SHEET STRUCTURE

You'll end up with six tabs. The good news: the script builds them all for you automatically in Section 4. Here's what each one is for so you understand the layout.

Materials tab — your list of items to be priced. Columns: Item, Description, Qty, Unit. Put one material per row under the header. Give each row a unique name in the Item column — vendor prices are matched back to your materials by that name, so two rows sharing an Item name would collide in the Comparison grid.

Vendors tab — who you're requesting pricing from. Columns: Vendor Name, Category, Email. One vendor per row. The Email column must contain a valid email address or that row is skipped.

Config tab — your settings and email wording, stored as simple Key/Value pairs. Keys: ProjectName, ContractorEmail, DueDate, EmailSubject, EmailBody. The subject and body support placeholders in double curly braces that get swapped in automatically: `{{ProjectName}}` , `{{VendorName}}` , `{{DueDate}}` , and `{{Link}}` . The `{{Link}}` placeholder is required in your body — it's where the vendor's unique pricing link gets inserted.

Responses tab — the system's private logbook. It's hidden automatically and you don't need to touch it. It stores each vendor's unique token, status, and submitted pricing.

Tracking tab — your dashboard. Shows every vendor, their status (blank = not yet sent, SENT, RECEIVED, or FAILED if Gmail rejected the address), when you sent, and when they responded. Repainted every time you refresh or send.

Comparison tab — your bid-leveling grid. Materials run down the side, vendors run across the top, and each vendor's submitted price drops into the right cell. The lowest price in each row is highlighted green so the best number per line item jumps out. Rebuilt every time a vendor submits, or on demand from the menu.

SECTION 04

INSTALLING THE SCRIPT

Open your Google Sheet. Click Extensions in the top menu, then Apps Script. A new tab opens with a code editor. Delete anything already in the file called `Code.gs`. Paste in the full `Code.gs` block below. Then click the + next to "Files," choose HTML, name it exactly `Form` (capital F, no extension), delete its contents, and paste in the `Form.html` block below. Click the save icon (or press Ctrl/Cmd+S). Close the Apps Script tab and reload your Google Sheet. You'll now see a new menu called Auto-RFP at the top.

Click Auto-RFP → Set Up Sheets (first time). This is where the first authorization happens — see the next section.

Code.gs

```
/**
 * AUTO-RFP ENGINE — TakeoffMonkey
 * Turns a Google Sheet into an automated pricing-request (RFP) machine.
 *
 * Tabs (auto-built by Set Up Sheets):
 * Materials - Item | Description | Qty | Unit (one material per row)
 * Vendors - Vendor Name | Category | Email (one vendor per row)
 * Config - Key | Value (settings + email wording)
 * Responses - hidden logbook (token, status, raw pricing JSON)
 * Tracking - dashboard (who you sent to, who replied, when)
 * Comparison - bid-leveling grid (materials down the side, vendors across the top)
 *
 * Install: paste this into Code.gs, create an HTML file named "Form",
 * paste Form.html into it, save, reload the sheet. Then use the Auto-RFP menu.
 */

// ----- CONFIG CONSTANTS -----
var SHEET_MATERIALS = 'Materials';
var SHEET_VENDORS = 'Vendors';
var SHEET_CONFIG = 'Config';
var SHEET_RESPONSES = 'Responses';
var SHEET_TRACKING = 'Tracking';
var SHEET_COMPARISON = 'Comparison';

var NUDGE_AFTER_HOURS = 48; // change this to adjust nudge timing

// Brand-ish colors for headers
var COLOR_HEADER_BG = '#00391F'; // Jungle Green
var COLOR_HEADER_FG = 'FFFFFF';
var COLOR_ACCENT = '#F5ED60'; // Banana
```



```

// Status / highlight fills (used by Tracking and the Comparison "lowest price" cell).
var COLOR_LOW_PRICE_BG = '#D7F5DD'; // green - lowest price per row in Comparison
var COLOR_RECEIVED_BG = '#D7F5DD'; // green - vendor submitted
var COLOR_SENT_BG = '#FFF6C8'; // amber - sent, awaiting reply
var COLOR_FAILED_BG = '#FBD6D2'; // red - send failed

// Per-execution reentrancy flag so a locked function that calls another locked
// function (e.g. submitPricing -> buildComparison) doesn't try to take the lock twice.
var _LOCK_HELD = false;

// ----- MENU -----
function onOpen() {
  SpreadsheetApp.getUi()
    .createMenu('Auto-RFP')
    .addItem('Set Up Sheets (first time)', 'setupSheets')
    .addSeparator()
    .addItem('Send RFPs to Vendors', 'sendRFPs')
    .addItem('Refresh Tracking', 'refreshTracking')
    .addItem('Build / Refresh Comparison', 'buildComparison')
    .addItem('Nudge Non-Responders', 'nudgeNonResponders')
    .addToUi();
}

// ----- SETUP -----
function setupSheets() {
  var ss = SpreadsheetApp.getActiveSpreadsheet();

  // Materials
  var mat = getOrCreateSheet(ss, SHEET_MATERIALS);
  if (mat.getLastRow() === 0) {
    setHeader_(mat, ['Item', 'Description', 'Qty', 'Unit']);
    mat.getRange(2, 1, 1, 4).setValues([[Example: 4" PVC Pipe', 'SDR-35 sewer pipe', '500', 'LF']]);
  }

  // Vendors
  var ven = getOrCreateSheet(ss, SHEET_VENDORS);
  if (ven.getLastRow() === 0) {
    setHeader_(ven, ['Vendor Name', 'Category', 'Email']);
    ven.getRange(2, 1, 1, 3).setValues([[Example Supply Co', 'Pipe & Fittings', 'sales@example.com']]);
  }

  // Config
  var cfg = getOrCreateSheet(ss, SHEET_CONFIG);
  if (cfg.getLastRow() === 0) {
    setHeader_(cfg, ['Key', 'Value']);
    var defaults = [
      ['ProjectName', 'Sample Project'],
      ['ContractorEmail', Session.getEffectiveUser().getEmail()],
      ['DueDate', formatDate_(new Date(Date.now()) + 7 * 24 * 3600 * 1000)],
      ['EmailSubject', 'Request for Pricing - {{ProjectName}}'],
      ['EmailBody',
        'Hi {{VendorName}},\n\n' +
        'We\'re requesting pricing for {{ProjectName}}. Please click your private link below to ' +
        'see the materials list and enter your prices. It takes just a couple minutes.\n\n' +
        '{{Link}}\n\n' +
        'Please submit by {{DueDate}}. Thanks!\n\n' +
        '— TakeoffMonkey'
      ];
    cfg.getRange(2, 1, defaults.length, 2).setValues(defaults);
  }
  cfg.setColumnWidth(1, 160);
  cfg.setColumnWidth(2, 520);

  // Responses (hidden)
  var resp = getOrCreateSheet(ss, SHEET_RESPONSES);
  if (resp.getLastRow() === 0) {
    setHeader_(resp, ['Token', 'Vendor Name', 'Email', 'Status', 'Sent At', 'Responded At', 'Pricing JSON']);
  }
}

```

```

resp.hideSheet();

// Tracking
var trk = getOrCreateSheet_(ss, SHEET_TRACKING);
trk.clear();
setHeader_(trk, ['Vendor Name', 'Category', 'Email', 'Status', 'Sent At', 'Responded At']);

// Comparison
getOrCreateSheet_(ss, SHEET_COMPARISON);

// Order tabs sensibly
reorderSheets_(ss, [SHEET_MATERIALS, SHEET_VENDORS, SHEET_CONFIG, SHEET_TRACKING, SHEET_COMPARISON]);

SpreadsheetApp.getUi().alert(
  'Setup complete.\n\n' +
  'Six tabs created: Materials, Vendors, Config, Tracking, Comparison, plus a hidden Responses log.\n\n' +
  'Next: fill in Materials + Vendors, set your Config, then DEPLOY the web app ' +
  '(Deploy > New deployment > Web app > Execute as Me > Who has access: Anyone), ' +
  'then use Send RFPs to Vendors.');
```

}

```

// ----- SEND RFPs -----
function sendRFPs() {
  withScriptLock(function () {
    var webAppUrl = getWebAppUrl_();
    if (!webAppUrl) {
      SpreadsheetApp.getUi().alert(
        'You must deploy the web app first.\n\n' +
        'Deploy > New deployment > Web app > Execute as: Me > Who has access: Anyone > Deploy.\n\n' +
        'Then open the Web app URL once in your browser, and run Send RFPs again.');
```

return;

```

    }

    var cfg = readConfig_();
    if (cfg.EmailBody.indexOf('{{Link}}') === -1) {
      SpreadsheetApp.getUi().alert('Your EmailBody must contain {{Link}} so the vendor link can be inserted. Add it
        in the Config tab.');
```

return;

```

    }

    var vendors = readVendors_();
    if (vendors.length === 0) {
      SpreadsheetApp.getUi().alert('No vendors found. Add at least one vendor (with a valid email) on the Vendors
        tab.');
```

return;

```

    }
    if (readMaterials_().length === 0) {
      SpreadsheetApp.getUi().alert('No materials found. Add at least one item on the Materials tab before sending -
        vendors would otherwise open an empty pricing form.');
```

return;

```

    }

    var resp = sheetOrThrow_(SHEET_RESPONSES);
    var existing = getResponseIndexByEmail_(); // email -> {row, token, status, sentAt}
    var now = new Date();
    var sent = 0, skippedInvalid = 0, skippedReceived = 0, skippedAlreadySent = 0, skippedDuplicate = 0, failedSend =
      0;
    var quotaStopped = false;
    var seen = {}; // emails handled in THIS run, so a duplicate Vendors row doesn't mint a second token

    for (var i = 0; i < vendors.length; i++) {
      var v = vendors[i];
      if (!isValidEmail_(v.email)) { skippedInvalid++; continue; }

      var key = v.email.toLowerCase();
      if (seen[key]) { skippedDuplicate++; continue; } // same email already processed this run
      seen[key] = true;

      var ex = existing[key];
```

```

    if (ex && ex.status === 'RECEIVED') { skippedReceived++; continue; }
    // Already emailed in a prior run - don't re-send (that's what Nudge is for). Skipping
    // here also means a quota-interrupted batch resumes at the UNSENT tail next day instead
    // of burning the day's quota re-mailing everyone it already reached.
    if (ex && ex.status === 'SENT' && ex.sentAt) { skippedAlreadySent++; continue; }

    // Stop cleanly when the daily mail quota is exhausted, BEFORE creating any row,
    // so a re-run tomorrow simply continues where we left off.
    if (MailApp.getRemainingDailyQuota() <= 0) { quotaStopped = true; break; }

    // Reuse an existing token (keeps old links alive); otherwise mint a new one.
    var token, rowIndex, isNew = false;
    if (ex) {
        token = ex.token;
        rowIndex = ex.row;
    } else {
        token = Utilities.getUuid();
        // Append as QUEUED - NOT 'SENT' - so a failed send never shows as delivered.
        resp.addRow([token, v.name, v.email, 'QUEUED', '', '', '']);
        rowIndex = resp.getLastRow();
        isNew = true;
    }

    var link = webAppUrl + '?token=' + encodeURIComponent(token);
    var subject = fillTemplate_(cfg.EmailSubject, { ProjectName: cfg.ProjectName, VendorName: v.name, DueDate:
        cfg.DueDate, Link: link });
    var body = fillTemplate_(cfg.EmailBody, { ProjectName: cfg.ProjectName, VendorName: v.name, DueDate:
        cfg.DueDate, Link: link });

    try {
        MailApp.sendEmail({ to: v.email, subject: subject, body: body });
        resp.getRange(rowIndex, 4).setValue('SENT');
        // Only stamp Sent At the first time, so re-sending doesn't reset the 48h nudge clock.
        if (isNew || !ex || !ex.sentAt) resp.getRange(rowIndex, 5).setValue(now);
        sent++;
    } catch (e) {
        failedSend++;
        if (isNew) resp.getRange(rowIndex, 4).setValue('FAILED'); // don't leave a false SENT; don't stamp Sent At
    }
}

refreshTracking();

var msg = 'Sent ' + sent + ' RFP email(s).';
var parts = [];
if (skippedInvalid) parts.push(skippedInvalid + ' invalid email');
if (skippedDuplicate) parts.push(skippedDuplicate + ' duplicate email');
if (skippedAlreadySent) parts.push(skippedAlreadySent + ' already sent (use Nudge to remind)');
if (skippedReceived) parts.push(skippedReceived + ' already received');
if (failedSend) parts.push(failedSend + ' send error');
if (parts.length) msg += '\nSkipped: ' + parts.join(', ') + '.';
if (quotaStopped) msg += '\n\n▲ Daily email limit reached - the remaining vendors were NOT sent. Run Send RFPs
    again tomorrow to finish the batch.';
SpreadsheetApp.getUi().alert(msg);
});
}

// ----- WEB APP: SHOW FORM -----
function doGet(e) {
    // Cache our own /exec URL so sendRFPs can build links automatically.
    // Only write when missing or changed, and never cache a /dev URL.
    try {
        var props = PropertiesService.getScriptProperties();
        var live = ScriptApp.getService().getUrl();
        if (live && /\s/.test(live) && props.getProperty('WEB_APP_URL') !== live) {
            props.setProperty('WEB_APP_URL', live);
        }
    } catch (err) {}
}

```

```

var token = e && e.parameter ? e.parameter.token : '';
var rec = token ? findResponseByToken_(token) : null;
var cfg = readConfig_();

var t = HtmlService.createTemplateFromFile('Form');
t.invalid = !rec;
t.alreadyDone = !(rec && rec.status === 'RECEIVED');
t.materials = (rec && !t.alreadyDone) ? readMaterials_() : [];
t.noMaterials = !(rec && !t.alreadyDone && t.materials.length === 0);
t.project = cfg.ProjectName;
t.vendorName = rec ? rec.vendorName : '';
t.dueDate = cfg.DueDate;
t.token = (rec && !t.alreadyDone) ? token : '';

return t.evaluate()
    .setTitle('Pricing Request')
    .addMetaTag('viewport', 'width=device-width, initial-scale=1');
// Note: no setXFrameOptionsMode(ALLOWALL) – keep the default same-origin frame protection.
}

// Called by the form via google.script.run
function submitPricing(payload) {
    // Validate the payload BEFORE touching the sheet so a malformed/hostile call
    // can't corrupt a row or burn the token.
    if (!payload || typeof payload.token !== 'string' || !Array.isArray(payload.prices)) {
        return { ok: false, message: 'Submission was malformed – please reload the page and try again.' };
    }
    var token = payload.token;
    var prices = normalizePrices_(payload.prices); // [{item, price, note}] coerced to strings

    // Server-side mirror of the form's check: never burn a token on an all-blank submit.
    var hasAny = prices.some(function (p) { return String(p.price || '').trim() !== ''; });
    if (!hasAny) { return { ok: false, message: 'Please enter at least one price before submitting.' }; }

    // Serialize the JSON up front and reject anything that can't fit a cell (50k char limit),
    // BEFORE the lock – so an oversized payload never strands the token on an empty RECEIVED row.
    var json = JSON.stringify(prices);
    if (json.length > 48000) {
        return { ok: false, message: 'This submission is too large to record. Please shorten your notes and try again.' };
    }

    // ATOMIC SECTION ONLY: re-check the token and commit the row, then release the lock fast.
    // The email + dashboard rebuilds happen AFTER the lock so they can't stretch the lock hold
    // long enough to time out a second vendor's concurrent submit.
    var committed = withScriptLock_(function () {
        var rec = findResponseByToken_(token); // re-read INSIDE the lock so the RECEIVED check is atomic
        if (!rec) { return { ok: false, message: 'This link is not valid.' }; }
        if (rec.status === 'RECEIVED') { return { ok: false, message: 'This request was already submitted. Thank you!' }; }
    });

    var resp = sheetOrThrow_(SHEET_RESPONSES);
    // Write the pricing payload FIRST and only flip to RECEIVED once it has landed, so a failed
    // write never burns the token. Then flush so a concurrent same-token submit sees RECEIVED.
    resp.getRange(rec.row, 7).setValue(json);
    resp.getRange(rec.row, 6).setValue(new Date());
    resp.getRange(rec.row, 4).setValue('RECEIVED');
    SpreadsheetApp.flush();
    return { ok: true, vendorName: rec.vendorName };
});
if (!committed.ok) { return committed; }

// Best-effort follow-ups OUTSIDE the lock – none may fail the vendor's submit, which already landed.
try {
    var cfg = readConfig_();
    var lines = prices.map(function (p) { return '• ' + p.item + ': ' + (p.price || '(no price)') + (p.note ? ' – ' + p.note : ''); });
    MailApp.sendEmail({
        to: cfg.ContractorEmail, // readConfig_ falls back to the script owner (the contractor) when blank
        subject: 'Pricing received from ' + committed.vendorName + ' – ' + cfg.ProjectName,
    });
}

```



```

        body: committed.vendorName + ' submitted pricing for ' + cfg.ProjectName + ':\n\n' + lines.join('\n')
    });
} catch (e) { try { console.error('Contractor copy email failed: ' + e); } catch (e2) {} }

try { refreshTracking(); } catch (e) { try { console.error('refreshTracking failed: ' + e); } catch (e2) {} }
try { buildComparison(); } catch (e) { try { console.error('buildComparison failed: ' + e); } catch (e2) {} }

return { ok: true, message: 'Pricing submitted. Thank you!' };
}

// ----- TRACKING -----
function refreshTracking() {
    withScriptLock_(function () {
        var trk = sheetOrThrow_(SHEET_TRACKING);
        var vendors = readVendors_();
        var index = getResponseIndexByEmail_();

        trk.clear();
        setHeader_(trk, ['Vendor Name', 'Category', 'Email', 'Status', 'Sent At', 'Responded At']);

        var rows = [];
        for (var i = 0; i < vendors.length; i++) {
            var v = vendors[i];
            var r = index[v.email.toLowerCase()];
            rows.push([
                v.name,
                v.category,
                v.email,
                r ? r.status : '',
                r && r.sentAt ? r.sentAt : '',
                r && r.respondedAt ? r.respondedAt : ''
            ]);
        }
        if (rows.length) trk.getRange(2, 1, rows.length, 6).setValues(rows);

        // Color the Status column
        for (var j = 0; j < rows.length; j++) {
            var statusCell = trk.getRange(j + 2, 4);
            if (rows[j][3] === 'RECEIVED') statusCell.setBackground(COLOR_RECEIVED_B6);
            else if (rows[j][3] === 'SENT') statusCell.setBackground(COLOR_SENT_B6);
            else if (rows[j][3] === 'FAILED') statusCell.setBackground(COLOR_FAILED_B6);
        }
        trk.autoResizeColumns(1, 6);
    });
}

// ----- COMPARISON (bid leveling) -----
function buildComparison() {
    withScriptLock_(function () {
        var ss = SpreadsheetApp.getActiveSpreadsheet();
        var cmp = getOrCreateSheet_(ss, SHEET_COMPARISON);
        cmp.clear();

        var materials = readMaterials_();
        var received = getReceivedResponses_(); // [{vendorName, prices:[{item,price,note}]}]

        // Header row: Item | Qty | Unit | <Vendor A> | <Vendor B> ...
        // Disambiguate vendors that happen to share a name so columns are readable.
        var nameCounts = {};
        received.forEach(function (r) { nameCounts[r.vendorName] = (nameCounts[r.vendorName] || 0) + 1; });
        var nameSeen = {};
        var vendorHeaders = received.map(function (r) {
            if (nameCounts[r.vendorName] > 1) {
                nameSeen[r.vendorName] = (nameSeen[r.vendorName] || 0) + 1;
                return r.vendorName + ' (' + nameSeen[r.vendorName] + ')';
            }
            return r.vendorName;
        });
        var header = ['Item', 'Qty', 'Unit'].concat(vendorHeaders);
    });
}

```

```

// A fresh sheet has only 26 columns; with many vendors the header would exceed the grid
// and getRange()/setValues() would throw. Widen the sheet first.
if (cmp.getMaxColumns() < header.length) {
  cmp.insertColumnsAfter(cmp.getMaxColumns(), header.length - cmp.getMaxColumns());
}
setHeader_(cmp, header);

if (materials.length === 0) {
  cmp.getRange(2, 1).setValue('Add materials on the Materials tab, then refresh.');
```

```

  return;
}

// Build a quick lookup: vendor -> {item -> price}
var lookup = received.map(function (r) {
  var m = {};
  (r.prices || []).forEach(function (p) { m[String(p.item).trim()] = p.price; });
  return m;
});
var matched = received.map(function () { return {}; }); // track which submitted items matched a material

var out = [];
for (var i = 0; i < materials.length; i++) {
  var keyName = String(materials[i].item).trim();
  var row = [materials[i].item, materials[i].qty, materials[i].unit];
  for (var k = 0; k < lookup.length; k++) {
    var val = lookup[k][keyName];
    if (val !== undefined) matched[k][keyName] = true;
    row.push(safeCell_(val != null ? val : '')); // safeCell_ neutralizes formula injection from vendor input
  }
  out.push(row);
}

// Surface any vendor-submitted prices whose item is NOT on the current Materials list
// (renamed or removed after sending) so prices are never silently dropped.
var unmatched = {};
for (var u = 0; u < received.length; u++) {
  (received[u].prices || []).forEach(function (p) {
    var ik = String(p.item).trim();
    if (ik && !matched[u][ik]) unmatched[ik] = true;
  });
}
var unmatchedItems = Object.keys(unmatched);
if (unmatchedItems.length) {
  out.push(['▲ Submitted for items no longer on the Materials list:', '', ''].concat(received.map(function () {
    return ''; })));
  unmatchedItems.forEach(function (ik) {
    // ik is a VENDOR-supplied item label – neutralize formula injection here too, not just on prices.
    var urow = [safeCell_(ik), '', ''];
    for (var k2 = 0; k2 < lookup.length; k2++) {
      var uval = lookup[k2][ik];
      urow.push(safeCell_(uval != null ? uval : ''));
    }
    out.push(urow);
  });
}

if (out.length) cmp.getRange(2, 1, out.length, header.length).setValues(out);

// Highlight the lowest numeric price per MATERIAL row (vendor columns 4..N). Skip the unmatched section.
for (var r2 = 0; r2 < materials.length; r2++) {
  var vals = out[r2].slice(3);
  var nums = vals.map(function (x) { var n = parseFloat(String(x).replace(/^[^0-9.\\-]/g, '')); return isNaN(n) ?
    null : n; });
  var min = null;
  for (var c = 0; c < nums.length; c++) {
    if (nums[c] !== null && (min === null || nums[c] < min)) min = nums[c];
  }
  // Highlight EVERY vendor tied for the lowest price, not just the leftmost.
  if (min !== null) {
```

```

        for (var c2 = 0; c2 < nums.length; c2++) {
            if (nums[c2] === min) cmp.getRange(r2 + 2, 4 + c2).setBackground(COLOR_LOW_PRICE_BG).setFontWeight('bold');
        }
    }
    cmp.autoResizeColumns(1, header.length);
    cmp.setFrozenColumns(1);
    cmp.setFrozenRows(1);
});
}

// ----- NUDGE -----
function nudgeNonResponders() {
    withScriptLock_(function () {
        var webAppUrl = getWebAppUrl_();
        if (!webAppUrl) { SpreadsheetApp.getUi().alert('Deploy the web app first (see Section 6), then open its URL once.');
```

once.');

```

        return; }

        var resp = sheetOrThrow_(SHEET_RESPONSES);
        var data = resp.getDataRange().getValues();
        var cfg = readConfig_();
        var now = new Date();
        var nudged = 0;

        for (var i = 1; i < data.length; i++) {
            var token = data[i][0], vendorName = data[i][1], email = data[i][2], status = data[i][3], sentAt = data[i][4];
            if (status !== 'SENT' || !sentAt) continue; // only nudge vendors who were actually sent and haven't responded
            var hours = (now - new Date(sentAt)) / 3600000;
            if (hours < NUDGE_AFTER_HOURS) continue;

            var link = webAppUrl + '?token=' + encodeURIComponent(token);
            var subject = 'Reminder: ' + fillTemplate_(cfg.EmailSubject, { ProjectName: cfg.ProjectName, VendorName: vendorName, DueDate: cfg.DueDate, Link: link });
            var body = 'Hi ' + vendorName + ',\n\nJust a friendly nudge on our pricing request for ' + cfg.ProjectName + '. Your private link is still active:\n\n' + link + '\n\nWe\'d appreciate your numbers by ' + cfg.DueDate + '. Thanks!\n\n- TakeoffMonkey';
            try { MailApp.sendEmail({ to: email, subject: subject, body: body }); nudged++; } catch (e) {}
        }
        SpreadsheetApp.getUi().alert('Nudged ' + nudged + ' non-responder(s) (sent more than ' + NUDGE_AFTER_HOURS + 'h ago, not yet received).');
    });
}

// ----- HELPERS -----
function getOrCreateSheet_(ss, name) {
    var sh = ss.getSheetByName(name);
    if (!sh) sh = ss.insertSheet(name);
    return sh;
}

// Fetch a required tab, or fail with a clear instruction instead of a cryptic null-deref
// (e.g. "Cannot read properties of null") if the contractor renamed or deleted it.
function sheetOrThrow_(name) {
    var sh = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(name);
    if (!sh) throw new Error('The "' + name + '" tab is missing. Run Auto-RFP → Set Up Sheets to recreate it.');
```

return sh;

```

}

function setHeader_(sh, headers) {
    sh.getRange(1, 1, 1, headers.length).setValues([headers])
        .setBackground(COLOR_HEADER_BG).setFontColor(COLOR_HEADER_F6).setFontWeight('bold');
    sh.setFrozenRows(1);
}

function reorderSheets_(ss, order) {
    for (var i = 0; i < order.length; i++) {
        var sh = ss.getSheetByName(order[i]);
        if (sh) { ss.setActiveSheet(sh); ss.moveActiveSheet(i + 1); }
    }
}

```

```

}

function readConfig_() {
  var sh = sheetOrThrow_(SHEET_CONFIG);
  var data = sh.getDataRange().getValues();
  var cfg = {};
  for (var i = 1; i < data.length; i++) {
    if (data[i][0]) cfg[String(data[i][0]).trim()] = data[i][1];
  }
  // sensible fallbacks
  cfg.ProjectName = cfg.ProjectName || 'Project';
  // getEffectiveUser() = the script OWNER (the contractor). getActiveUser() returns '' for an
  // anonymous web-app visitor, which is exactly when the contractor-copy email is sent - so the
  // active user would silently drop the copy. The owner is always the right recipient here.
  cfg.ContractorEmail = cfg.ContractorEmail || Session.getEffectiveUser().getEmail();
  // If the contractor typed a real date into the Config cell, Sheets stores it as a Date object,
  // which renders as an ugly "Thu Jun 26 2026 00:00:00 GMT..." string in emails. Format it.
  if (cfg.DueDate instanceof Date) cfg.DueDate = formatDate_(cfg.DueDate);
  else cfg.DueDate = cfg.DueDate || '';
  cfg.EmailSubject = cfg.EmailSubject || 'Request for Pricing - {{ProjectName}}';
  cfg.EmailBody = cfg.EmailBody || 'Hi {{VendorName}}, please price our materials: {{Link}} (due {{DueDate}}).';
  return cfg;
}

function readVendors_() {
  var sh = sheetOrThrow_(SHEET_VENDORS);
  var data = sh.getDataRange().getValues();
  var out = [];
  for (var i = 1; i < data.length; i++) {
    var name = data[i][0], cat = data[i][1], email = data[i][2];
    if (!name && !email) continue;
    out.push({ name: String(name || '').trim(), category: String(cat || '').trim(), email: String(email || '').trim() });
  }
  return out;
}

function readMaterials_() {
  var sh = sheetOrThrow_(SHEET_MATERIALS);
  var data = sh.getDataRange().getValues();
  if (data.length === 0) return [];
  var headers = data[0];
  var out = [];
  for (var i = 1; i < data.length; i++) {
    if (!data[i][0]) continue; // skip rows with no Item (Item is the line's identity/label)
    var obj = { item: String(data[i][0]).trim(), description: data[i][1] || '', qty: data[i][2] || '', unit:
      data[i][3] || '', extra: {} };
    // carry any extra columns beyond the standard 4 (not shown on the vendor form)
    for (var c = 4; c < headers.length; c++) { if (headers[c]) obj.extra[headers[c]] = data[i][c]; }
    out.push(obj);
  }
  return out;
}

function findResponseByToken_(token) {
  var sh = sheetOrThrow_(SHEET_RESPONSES);
  var data = sh.getDataRange().getValues();
  for (var i = 1; i < data.length; i++) {
    if (data[i][0] === token) {
      return { row: i + 1, token: data[i][0], vendorName: data[i][1], email: data[i][2], status: data[i][3], sentAt:
        data[i][4], respondedAt: data[i][5] };
    }
  }
  return null;
}

function getResponseIndexByEmail_() {
  var sh = sheetOrThrow_(SHEET_RESPONSES);
  var data = sh.getDataRange().getValues();

```

```

var idx = {};
for (var i = 1; i < data.length; i++) {
  var email = String(data[i][2] || '').toLowerCase();
  if (email) idx[email] = { row: i + 1, token: data[i][0], status: data[i][3], sentAt: data[i][4], respondedAt:
    data[i][5] };
}
return idx;
}

function getReceivedResponses_() {
  var sh = sheetOrThrow_(SHEET_RESPONSES);
  var data = sh.getDataRange().getValues();
  var out = [];
  for (var i = 1; i < data.length; i++) {
    if (data[i][3] === 'RECEIVED' && data[i][6]) {
      var prices = [];
      try { prices = JSON.parse(data[i][6]); } catch (e) {}
      if (!Array.isArray(prices)) prices = []; // guard against a non-array JSON value
      out.push({ vendorName: data[i][1], prices: prices });
    }
  }
  return out;
}

function getWebAppUrl_() {
  var url = '';
  try { url = ScriptApp.getService().getUrl() || ''; } catch (e) {}
  // Prefer a live /exec URL. If we got null or a /dev (HEAD) URL, fall back to the
  // /exec URL cached by doGet - never hand a /dev URL to vendors (it requires login).
  if (!/\s$/.test(url)) {
    try { url = PropertiesService.getScriptProperties().getProperty('WEB_APP_URL') || ''; } catch (e2) {}
  }
  return /\s$/.test(url) ? url : '';
}

function fillTemplate_(tpl, map) {
  return String(tpl).replace(/\{(\w+)\}\}/g, function (m, k) { return (map[k] !== undefined && map[k] !== null) ?
    map[k] : ''; });
}

function isValidEmail_(email) {
  return /^[^@]+@[^@]+\.[^@]+\s$/.test(String(email || '').trim());
}

function formatDate_(d) {
  return Utilities.formatDate(d, Session.getScriptTimeZone(), 'MMM d, yyyy');
}

// Neutralize spreadsheet formula injection: a vendor-supplied value that starts with a
// formula trigger (= + - @) or a leading control char is written as text by prefixing an
// apostrophe, so it can't execute as a live formula in the contractor's sheet. The =/+/-/@
// set matches OWASP's CSV-injection guidance.
function safeCell_(s) {
  s = (s == null ? '' : String(s));
  return /^[=+-@\t\r\n]/.test(s) ? "'" + s : s;
}

// Collapse any line breaks to spaces so a multi-line value can't forge extra rows in the
// contractor's copy email or sprawl across a cell.
function oneLine_(s) {
  return String(s == null ? '' : s).replace(/\r\n/g, ' ');
}

// Coerce a submitted prices payload to a safe, bounded array of {item, price, note} strings.
function normalizePrices_(arr) {
  if (!Array.isArray(arr)) return [];
  return arr.slice(0, 1000).map(function (p) {
    p = p || {};
    return {

```

```

        item: oneLine_(p.item ).slice(0, 300),
        price: oneLine_(p.price).slice(0, 100),
        note: oneLine_(p.note ).slice(0, 500)
    };
    });
}

// Run fn() while holding the script lock. Reentrant within one execution: nested locked
// calls (submitPricing -> buildComparison) just run fn() instead of re-locking (which would
// deadlock). Serializes concurrent web-app submits and menu rebuilds against each other.
function withScriptLock(fn) {
    if (_LOCK_HELD) return fn();
    var lock = LockService.getScriptLock();
    try {
        lock.waitLock(30000);
    } catch (e) {
        // Turn the raw "could not obtain lock" exception into a friendly message - shown to the
        // contractor as a toast, or returned to a vendor via google.script.run's failure handler.
        throw new Error('The sheet is busy right now - please wait a moment and try again.');
```

Form.html

```

<!DOCTYPE html>
<html>
<head>
    <base target="_top">
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        :root {
            --jungle: #00391F;
            --banana: #F5ED60;
            --concrete: #F1F2F2;
            --ink: #231F20;
        }
        * { box-sizing: border-box; }
        body {
            font-family: 'Montserrat', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
            margin: 0; background: var(--concrete); color: var(--ink);
        }
        .wrap { max-width: 760px; margin: 0 auto; padding: 24px 16px 64px; }
        .card { background: #fff; border-radius: 14px; box-shadow: 0 2px 14px rgba(0,0,0,.08); overflow: hidden; }
        .header { background: var(--jungle); color: #fff; padding: 22px 24px; }
        .header h1 { margin: 0 0 4px; font-size: 20px; }
        .header p { margin: 0; opacity: .9; font-size: 14px; }
        .body { padding: 20px 24px; }
        .meta { font-size: 14px; color: #444; margin-bottom: 18px; }
        .meta b { color: var(--ink); }
        table { width: 100%; border-collapse: collapse; }
        th, td { text-align: left; padding: 10px 8px; border-bottom: 1px solid #eee; font-size: 14px; vertical-align: top; }
        th { background: #fafafa; font-weight: 700; }
        td.num { white-space: nowrap; }
        input[type=text] {
            width: 100%; padding: 9px 10px; border: 1px solid #cfcfcf; border-radius: 8px; font-size: 14px;
        }
        .price-col { width: 130px; }
```

```

.note-col { width: 200px; }
.actions { margin-top: 22px; display: flex; align-items: center; gap: 14px; flex-wrap: wrap; }
button {
  background: var(--jungle); color: #fff; border: 0; border-radius: 10px;
  padding: 13px 26px; font-size: 16px; font-weight: 700; cursor: pointer;
}
button:disabled { opacity: .5; cursor: default; }
.msg { font-size: 14px; }
.msg.ok { color: #0b7a2f; font-weight: 700; }
.msg.err { color: #b00020; font-weight: 700; }
.notice { padding: 40px 24px; text-align: center; }
.notice h2 { color: var(--jungle); }
.pill { display:inline-block; background: var(--banana); color: var(--ink); font-weight:700; padding: 2px 10px;
  border-radius: 999px; font-size: 12px; }
@media (max-width: 520px) {
  .price-col, .note-col { width: auto; }
  th:nth-child(2), td:nth-child(2) { display: none; } /* hide Description on tiny screens */
}
</style>
</head>
<body data-token="<?= token ?">
  <div class="wrap">
    <div class="card">

      <? if (invalid) { ?>
        <div class="notice">
          <h2>This link is not valid</h2>
          <p>The pricing request couldn't be found. It may have expired or been replaced.
            Please contact the company that sent it for a fresh link.</p>
        </div>

      <? } else if (alreadyDone) { ?>
        <div class="notice">
          <h2>Already submitted</h2>
          <p>We've already received your pricing for <b><?= project ?></b>. Thank you!
            If you need to make a change, contact the requester for a new link.</p>
        </div>

      <? } else if (noMaterials) { ?>
        <div class="notice">
          <h2>Pricing request not ready</h2>
          <p>The materials list for <b><?= project ?></b> hasn't been added yet.
            Please contact the company that sent this link for an updated request.</p>
        </div>

      <? } else { ?>
        <div class="header">
          <h1>Pricing Request – <?= project ?></h1>
          <p>For <b><?= vendorName ?></b><? if (dueDate) { ?> · due <?= dueDate ?><? } ?></p>
        </div>
        <div class="body">
          <div class="meta">
            Enter your unit price for each item below, then click <b>Submit Pricing</b>.
            Leave the note blank unless you need to flag something (lead time, substitution, etc.).
            <span class="pill">Takes ~2 minutes</span>
          </div>

          <table id="mtable">
            <thead>
              <tr>
                <th>Item</th>
                <th>Description</th>
                <th class="num">Qty</th>
                <th class="num">Unit</th>
                <th class="price-col">Your Price</th>
                <th class="note-col">Note (optional)</th>
              </tr>
            </thead>
            <tbody>

```

```

        <? for (var i = 0; i < materials.length; i++) { ?>
        <tr>
        <td data-item="<?= materials[i].item ?>"><?= materials[i].item ?></td>
        <td><?= materials[i].description ?></td>
        <td class="num"><?= materials[i].qty ?></td>
        <td class="num"><?= materials[i].unit ?></td>
        <td class="price-col"><input type="text" class="price" inputmode="decimal" placeholder="$"
            aria-label="Price for <?= materials[i].item ?>"></td>
        <td class="note-col"><input type="text" class="note" placeholder="" aria-label="Note for <?=
            materials[i].item ?>"></td>
        </tr>
        <? } ?>
    </tbody>
</table>

    <div class="actions">
        <button id="submitBtn" onclick="submitForm()">Submit Pricing</button>
        <span id="msg" class="msg"></span>
    </div>
</div>
<? } ?>

</div>
</div>

<script>
    // Read the token from the body's data attribute (server-escaped for HTML-attribute
    // context) rather than inlining it into this script block.
    var TOKEN = document.body.getAttribute('data-token') || '';

    function showSubmitted(text) {
        document.querySelector('.card').innerHTML =
            '<div class="notice"><h2>Pricing submitted</h2><p>' + text + ' You can close this window.</p></div>';
    }

    function submitForm() {
        var rows = document.querySelectorAll('#mtable tbody tr');
        var prices = [];
        rows.forEach(function (tr) {
            var item = tr.querySelector('td[data-item]').getAttribute('data-item');
            var price = tr.querySelector('input.price').value.trim();
            var note = tr.querySelector('input.note').value.trim();
            prices.push({ item: item, price: price, note: note });
        });

        var hasAny = prices.some(function (p) { return p.price !== ''; });
        var msg = document.getElementById('msg');
        if (!hasAny) {
            msg.className = 'msg err';
            msg.textContent = 'Please enter at least one price before submitting.';
            return;
        }

        var btn = document.getElementById('submitBtn');
        btn.disabled = true;
        msg.className = 'msg';
        msg.textContent = 'Submitting...';

        google.script.run
            .withSuccessHandler(function (res) {
                if (res && res.ok) {
                    showSubmitted(res.message);
                } else if (res && res.message && /already submitted/i.test(res.message)) {
                    // A double-tap or stale retry – the pricing already landed, so reassure rather than alarm.
                    showSubmitted(res.message);
                } else {
                    msg.className = 'msg err';
                    msg.textContent = (res && res.message) ? res.message : 'Something went wrong. Please try again.';
                    btn.disabled = false;
                }
            })
    }

```



```

    }
  })
  .withFailureHandler(function (err) {
    msg.className = 'msg err';
    msg.textContent = 'Network error – please try again. (' + (err && err.message ? err.message : 'unknown') +
    ')';
    btn.disabled = false;
  })
  .submitPricing({ token: TOKEN, prices: prices });
}
</script>
</body>
</html>

```

SECTION 05

THE FIRST AUTHORIZATION (IMPORTANT — DON'T PANIC)

The very first time you run anything, Google asks permission for the script to send email and edit your sheet on your behalf. The screens look alarming but this is normal for any custom script you install yourself.

You'll see "Authorization required." Click Review permissions. Choose your Google account. Then you'll likely see a red-ish warning: "Google hasn't verified this app." This is expected — it's your own script, not a published app, so Google hasn't reviewed it. Click the small Advanced link at the bottom left. Then click Go to [project name] (unsafe). The word "unsafe" is just Google's generic language for any unverified script; you wrote/installed this one yourself. On the next screen, review the permissions (send email as you, manage this spreadsheet) and click Allow.

You only do this once. After that, the buttons just work.

SECTION 06

DEPLOYING THE VENDOR PRICING WEB FORM

The vendor link points to a small web app served from your script. You need to publish it once.

In the Apps Script editor, click the blue Deploy button (top right), then New deployment. Click the gear icon next to "Select type" and choose Web app. Under settings: Description can be anything (e.g. "RFP form"). "Execute as" → Me. "Who has access" → Anyone. This is required so vendors who aren't logged into your Google account can open the form. Click Deploy. Authorize again if prompted (same steps as Section 5). Copy the Web app URL it gives you — then open it once in your browser. The page will say "This link is not valid" — that's expected and exactly what you want: you opened it without a vendor token, and that first visit lets the script cache its own address so your Send button can build vendor

links automatically.

Important: every time you change the code, you must redeploy for vendors to get the new version. Do this via Deploy → Manage deployments → (pencil/edit icon) → Version: New version → Deploy. If you create a brand-new deployment instead, the URL changes and old links break — so always edit the existing one.

SECTION 07

SENDING YOUR FIRST BATCH OF RFPs

Fill in your Materials tab and Vendors tab — and replace or delete the example rows that setup added (otherwise the placeholder vendor counts as a real send). Open the Config tab and set ProjectName, DueDate, your EmailSubject and EmailBody (keep `{{Link}}` somewhere in the body). Then click Auto-RFP → Send RFPs to Vendors. The script loops your vendor list, skips any row with a blank or invalid email, generates a unique private link per vendor, sends each a personalized Gmail, and logs them all. You'll get a popup telling you how many went out. Switch to the Tracking tab to see them all marked SENT.

Running Send again is safe: it only emails vendors it hasn't already contacted (so a batch interrupted by the daily limit resumes cleanly the next day, and nobody gets a duplicate). To re-contact vendors who haven't replied, use Nudge (Section 9), not Send.

SECTION 08

READING THE TRACKING TAB

The Tracking tab is your dashboard: Vendor Name, Category, Email, Status, Sent At, Responded At. Status is SENT (amber) until the vendor submits, then flips to RECEIVED (green) with a timestamp; a bad address shows FAILED (red). Click Auto-RFP → Refresh Tracking any time to repaint it with the latest. When a vendor submits, you also get an email copy of their pricing automatically, and the full pricing is stored in the hidden Responses tab.

SECTION 09

NUDGING NON-RESPONDERS

Click Auto-RFP → Nudge Non-Responders. The script finds every vendor still marked SENT whose request went out more than 48 hours ago, and re-sends them a friendly reminder with the same private link. Vendors who already submitted are never nudged. You'll get a popup with the count.



SECTION 10

READING THE COMPARISON TAB (BID LEVELING)

The Comparison tab is where you actually pick a winner. Materials run down the left (Item, Qty, Unit), and every vendor who has submitted gets their own column across the top. Each price drops into the matching cell, and the lowest number in each row is highlighted green so the best price per line item is obvious at a glance.

It rebuilds automatically every time a vendor submits, so it's always current. You can also force a rebuild any time with Auto-RFP → Build / Refresh Comparison — handy if you tweaked your materials list or just want a clean repaint. Only vendors who have submitted get a column, so a blank cell means that vendor didn't price that particular line — not that they haven't responded (non-responders simply have no column yet).

SECTION 11

TROUBLESHOOTING & LIMITS

Daily send limits: ~100 recipients/day on personal Gmail, ~1,500/day on Workspace. Exceeding it pauses sending until the next day. Send large lists in batches.

"You must deploy the web app first" popup: you ran Send before deploying — or you deployed but never opened the Web app URL. Do Section 6 (including opening the URL once), then send again.

Vendor says the link doesn't work / shows old form: you changed the code without redeploying. Go to Deploy → Manage deployments → edit → New version → Deploy.

Vendor sees "This link is not valid": the token isn't in the hidden Responses tab — usually because that vendor's row was deleted from it, or the link was hand-edited in transit. (Re-sending reuses the same token, so older copies of a vendor's link keep working — a re-send does not invalidate them.) Fix it by re-sending to that one vendor.

"unsafe" warning during auth: normal for self-installed scripts. Advanced → Go to project → Allow (Section 5).

Emails not arriving: check the vendor's spam folder and confirm the Email column has a valid address. The script silently skips invalid rows.

A vendor submitted twice: the form blocks resubmission — once a token is RECEIVED it shows an "already submitted" message and won't overwrite.



How private are the links? Each link carries a long random token and the form is reachable by anyone who has it (that's what lets vendors open it without a Google login). Treat the links like a building key: anyone the vendor forwards it to could enter pricing as them, and there's no expiry. For routine material RFPs this is fine; don't use it for anything you'd consider confidential.

SECTION 12

HOW TO CUSTOMIZE THIS

Change the email wording: edit EmailBody and EmailSubject in the Config tab. Use `{{VendorName}}` , `{{ProjectName}}` , `{{DueDate}}` , `{{Link}}` anywhere.

Add material columns: the vendor form always shows a fixed set of columns — Item, Description, Qty, Unit — plus the Your Price and Note inputs. Extra columns you add to the Materials tab are not shown to vendors and don't appear in the Comparison grid, so if vendors need more context (a spec, manufacturer, or location), put it in the Description column. (Developers: extra columns are read into each material's `extra` map in `readMaterials_` if you want to render them on the form.)

Filter which vendors get which materials: the simplest approach is one sheet per trade/category (duplicate the whole sheet). Note that each duplicated sheet is its own Apps Script project, so you'll deploy its web app once too (Section 6) before sending from it. For advanced users, add a Category match between the Vendors tab and a Category column on Materials, and filter the loop in `sendRFPs()` .

Change the nudge timing: at the top of `Code.gs` , change the `NUDGE_AFTER_HOURS` value (default 48).

Change the brand colors: the header colors, the Tracking status fills, and the green "lowest price" highlight (`COLOR_LOW_PRICE_BG`) all live in the `COLOR_*` constants at the top of `Code.gs` ; the form's colors live in the `:root` variables in `Form.html` .